PROJECT REPORT

on

FOOTBALL CLUB MANAGEMENT SYSTEM

A Comprehensive Web-Based Solution for Digitizing Football Club Operations

---

Submitted in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Computer Applications (BCA)

Submitted by:

Athul

Under the guidance of:

Faculty Guide

Department of Computer Applications

University Name

April 2026

CERTIFICATE

This is to certify that the project report entitled **"Football Club Management System"** submitted by **Athul** is a bonafide record of the project work carried out under my guidance and supervision in partial fulfillment of the requirements for the award of the degree of **Bachelor of Computer Applications (BCA)**.

Internal Guide
Department of Computer Applications

Head of Department
Department of Computer Applications

External Examiner

Date: **Place:**

DECLARATION

I hereby declare that the project report entitled **"Football Club Management System"** is an authentic record of my own work carried out as a requirement for the award of the degree of **Bachelor of Computer Applications (BCA)**. This work has not been submitted to any other university or institution for the award of any degree or diploma.

Athul

Date: April 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project.

I am deeply grateful to my project guide for the continuous support, invaluable guidance, and constructive feedback provided throughout the development of this project. The technical insights and mentorship were instrumental in shaping the system architecture and feature set.

I extend my heartfelt thanks to the **Head of the Department of Computer Applications** for providing the necessary infrastructure and academic environment that facilitated this work.

I also wish to thank the staff members and fellow students who participated in the User Acceptance Testing phase, providing the practical feedback that helped refine the system's usability and functionality.

Finally, I express my gratitude to my family and friends for their unwavering encouragement and moral support throughout the duration of this project.

Athul

ABSTRACT

The **Football Club Management System** is a comprehensive, full-stack web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) designed to digitize and streamline every operational facet of a professional football club. Traditional football club management relies on fragmented manual processes—spreadsheets for player statistics, paper forms for leave requests, phone calls for training notifications, and disconnected filing cabinets for contracts. This project replaces that paradigm with a centralized, real-time, role-based digital platform.

The system provides four distinct, interconnected dashboards for **Admin**, **Manager**, **Coach**, and **Player** roles, each equipped with tailored tools and live data powered by Socket.IO WebSocket communication. Key capabilities include user lifecycle management with immutable audit logging, fixture scheduling with tactical lineup building, training session management with attendance tracking, squad health monitoring with injury and fitness tracking, leave request workflows with real-time approval notifications, document vault with OCR scanning, and equipment inventory management.

Beyond standard CRUD operations, the application integrates four AI-powered features: an OCR document scanner, an AI chatbot, a document preparation assistant, and a text summarization engine. The system employs enterprise-grade security through JWT authentication, bcrypt password hashing, Helmet security headers, CORS policies, multi-tier rate limiting, and input sanitization. The cinematic, EA FC-inspired user interface features dark-mode glassmorphism, animated player cards, and dynamic page transitions.

The system was developed, tested through unit, integration, and user acceptance testing, and validated against 25 formal test cases—all passing successfully.

Keywords: MERN Stack, Football Club Management, Real-Time Communication, Socket.IO, Role-Based Access Control, AI Integration, MongoDB, React.js, Node.js

CONTENTS

1 INTRODUCTION **1**

1.1 Project Overview 1

1.2 Problem Statement 1

1.3 Project Objectives 2

1.4 Scope of the Project 2

2 SYSTEM SPECIFICATIONS **4**

2.1 Software Specification 4

2.2 Hardware Specification 5

3 SYSTEM ANALYSIS **6**

3.1 Existing System 6

3.2 Proposed System 6

3.3 Feasibility Study 7

3.3.1 Technical Feasibility 7

3.3.2 Economic Feasibility 7

3.3.3 Operational Feasibility 7

4 SYSTEM DESIGN **8**

4.1 Data Flow Diagrams 8

4.1.1 Level 0 — Context Diagram 8

4.1.2 Level 1 — Admin Module DFD 9

4.1.3 Level 1 — Manager Module DFD 9

4.1.4 Level 1 — Coach Module DFD 10

4.1.5 Level 1 — Player Module DFD 10

4.2 Entity Relationship Diagram 11

5 TABLE DESIGN **12**

5.1 Users Collection 12

5.2 Profiles Collection 12

5.3 Players Collection 13

5.4 ClubMemberships Collection 13

5.5 SeasonStats Collection 13

5.6 ArchivedPlayers Collection 13

5.7 Fixtures Collection 14

5.8 TrainingSessions Collection 14
5.9 Injuries Collection 14
5.10 LeaveRequests Collection 14
5.11 DisciplinaryActions Collection 14
5.12 Documents Collection 14
5.13 Inventories Collection 15
5.14 Settings Collection (Singleton) 15
5.15 SystemLogs Collection (Immutable) 15

6 MODULE DESIGN AND IMPLEMENTATION 16
6.1 Login Page 16
6.2 Admin Module 16
6.2.1 User Management 16
6.2.2 Player Archive 17
6.2.3 Club Settings 17
6.2.4 System Logs 17
6.3 Manager Module 17
6.3.1 Fixtures 17
6.3.2 Contracts 17
6.3.3 Document Vault 17
6.3.4 Inventory 17
6.3.5 Finance Dashboard 18
6.4 Coach Module 18
6.4.1 Tactical Board 18
6.4.2 Training Schedule 18
6.4.3 Squad Health 18
6.4.4 Discipline 18
6.4.5 Leave Approval 18
6.4.6 Performance Tracking 18
6.5 Player Module 18
6.5.1 Personalized Dashboard 19
6.5.2 Calendar 19
6.5.3 Leave Requests 19
6.6 AI-Powered Features 19
6.6.1 OCR Document Scanner 19
6.6.2 AI Document Preparation 19
6.6.3 AI Chatbot 19
6.6.4 Text Summarization 19

7 SECURITY MEASURES **20**

8 SYSTEM TESTING **22**

8.1 Unit Testing 22

8.2 Integration Testing 22

8.3 Test Cases 22

8.4 User Acceptance Testing 23

9 SCOPE FOR FUTURE ENHANCEMENT **25**

10 CONCLUSION **26**

LIST OF FIGURES

3.1 Three-Layer System Architecture 7

4.1 Level 0 — Context Diagram 9
4.2 Level 1 — Admin Module DFD 9
4.3 Level 1 — Manager Module DFD 10
4.4 Level 1 — Coach Module DFD 10
4.5 Level 1 — Player Module DFD 11
4.6 Entity Relationship Diagram — 15 Collections 11

LIST OF TABLES

1.1 Functional Scope of the System 3

2.1 Software Specification 4
2.2 Hardware Specification 5

4.1 DFD Symbol Definitions 8

5.1 Users Collection Schema 12
5.2 Profiles Collection Schema (Key Fields) 12
5.3 Players Collection Schema 13
5.4 Fixtures Collection Schema 14

7.1 Security Features Summary 20

8.1 System Test Cases 23
8.2 User Acceptance Testing Results 23

CHAPTER 1

INTRODUCTION

1.1 Project Overview

The Football Club Management System is a comprehensive, web-based application designed to replace traditional, paper-based football club operations. It digitizes and streamlines key operational and administrative workflows by providing four distinct, role-based interfaces: an **Admin Panel** for central control, a **Manager Panel** for scheduling and resources, a **Coach Panel** for tactical and squad management, and a **Player Panel** for personal dashboards and requests.

These modules function as a single, interconnected ecosystem built on the MERN stack (MongoDB, Express.js, React.js, Node.js). The system leverages Socket.IO for real-time WebSocket communication, ensuring that actions taken by one user role are instantly reflected for others. For example, when a coach approves a leave request, the player receives an immediate notification on their dashboard without requiring a page refresh.

The platform integrates four AI-powered capabilities—OCR document scanning, AI chatbot assistance, document preparation, and text summarization—to reduce manual labor and accelerate decision-making. Security is implemented through JWT-based authentication, bcrypt password hashing, role-based access control (RBAC), Helmet security headers, CORS policies, rate limiting, input sanitization, and an immutable audit trail.

1.2 Problem Statement

The traditional football club management system is fundamentally inefficient, relying on outdated, manual, and disconnected processes. The core challenges include:

- **Data Fragmentation:** Player statistics, injury records, and contract details are scattered across multiple spreadsheets with no single source of truth, leading to version conflicts and formula errors.
- **Communication Gaps:** Training schedules, match fixtures, and leave decisions are communicated via WhatsApp groups or phone calls, with no formal logging or acknowledgment system.
- **Administrative Overhead:** Coaches maintain paper-based lineup records, manually

track training attendance, and physically compile fitness reports—processes that are time-consuming and error-prone.

- **Security Vulnerabilities:** Sensitive player data is stored in unencrypted formats without access controls, creating compliance and accountability risks.
- **No Audit Trail:** There is no mechanism to track who modified which records and when, making it impossible to trace unauthorized changes.
- **Document Management Chaos:** Physical documents are prone to loss and damage, with no OCR or digital search capability.

1.3 Project Objectives

The project is guided by the following specific objectives:

1. **Centralized Operations Hub** — Build a unified web platform where Admin, Manager, Coach, and Player roles access only the tools relevant to their responsibilities.
2. **Real-Time Communication** — Leverage Socket.IO for live notifications, online-presence tracking, and instant data synchronization across all clients.
3. **AI-Augmented Workflows** — Integrate OCR scanning, AI chatbot, document preparation, and text summarization to reduce manual labor.
4. **Immutable Audit Trail** — Log every CREATE, UPDATE, DELETE, and LOGIN action in a tamper-proof system log.
5. **Cinematic UI/UX** — Deliver a visually stunning, EA FC-inspired interface with dark themes, glassmorphism, animated player cards, and page transitions.
6. **Comprehensive Player Lifecycle** — Support players from recruitment through active play to archival with full career snapshot preservation.
7. **Enterprise-Grade Security** — Protect all data with JWT authentication, bcrypt hashing, Helmet headers, CORS, rate limiting, and input sanitization.

1.4 Scope of the Project

The scope encompasses the following functional domains, as summarized in Table 1.1.

Table 1.1: Functional Scope of the System

Domain	Description
Admin Panel	User CRUD, player archival/reinstatement, club settings, immutable system logs
Manager Panel	Fixtures, contracts, document vault with OCR, inventory, finance dashboard
Coach Panel	Tactical board with lineup builder, training schedule, squad health, discipline, leave approval, performance tracking
Player Panel	Personal dashboard, event calendar, leave request submission
AI Features	OCR scanner, AI chatbot, document preparation, text summarization
Real-Time	Socket.IO notifications, online presence, live data synchronization
Security	JWT auth, bcrypt, Helmet, CORS, rate limiting, sanitization, immutable logs

Out of Scope: Payment gateway integration, video analysis, native mobile applications, multi-language support, biometric device integration, third-party football data APIs, multi-club/league management, and parent/guardian portals. These are identified as future enhancements.

CHAPTER 2

SYSTEM SPECIFICATIONS

2.1 Software Specification

The system is developed using a three-layer MERN architecture separating the Presentation, Logic, and Data layers. Table 2.1 details the software stack.

Table 2.1: Software Specification

Component	Technology	Version	Purpose
Frontend	React.js + Vite	18.x	SPA with component architecture
Routing	React Router DOM	6.x	Client-side navigation
State Mgmt	React Context API	—	AuthContext, SocketContext
Styling	Vanilla CSS + Tokens	—	Design system with custom properties
Typography	Google Fonts	—	Bebas Neue (headings), Inter (body)
Backend	Node.js + Express	18.x / 4.18.2	RESTful API server
Database	MongoDB + Mongoose	7.x / 8.0.0	NoSQL document store with ODM
Auth	JWT + bcrypt	9.0.2 / 5.1.1	Token auth + password hashing
Real-Time	Socket.IO	4.6.0	WebSocket communication
File Upload	Multer + Sharp	1.4.5 / 0.33.0	Multipart upload + image processing
Security	Helmet + CORS + Rate Limit	7.1.0 / 2.8.5 / 7.1.5	HTTP headers, CORS, throttling
Sanitization	sanitize-html + validator	2.11.0 / 13.11.0	XSS prevention, input validation
Testing	Jest + Supertest	29.7.0 / 6.3.3	Unit + integration testing

2.2 Hardware Specification

Table 2.2 outlines the hardware requirements for the server and client environments.

Table 2.2: Hardware Specification

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB or higher
Storage	10 GB free disk space	20 GB SSD
Display	1366 $\times$ 768	1920 $\times$ 1080 Full HD
Network	Broadband Internet	Stable 10 Mbps+
GPU	Integrated graphics	Dedicated GPU for animations
Browser	Chrome / Edge (latest)	Chrome / Edge (latest)

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

The existing system for football club management is a collection of traditional, manual, and disconnected processes:

- **Spreadsheet-Based Tracking:** Player statistics, injury records, and contract dates maintained in Excel/Google Sheets, leading to version conflicts and no access control.
- **Paper-Based Documentation:** Contracts and medical certificates stored as physical files, making retrieval slow and prone to loss.
- **Disconnected Communication:** Training schedules communicated via WhatsApp groups with no formal logging or acknowledgment.
- **No Role Separation:** All staff share the same access level, creating security risks.
- **No Audit Trail:** No record of who changed what and when.
- **No AI Assistance:** All document processing is entirely manual.
- **No Real-Time Updates:** Lineup changes and leave approvals require manual notification.

3.2 Proposed System

The proposed Football Club Management System addresses every limitation through a centralized, real-time, web-based platform with four interconnected role-based panels. Figure 3.1 illustrates the system architecture.

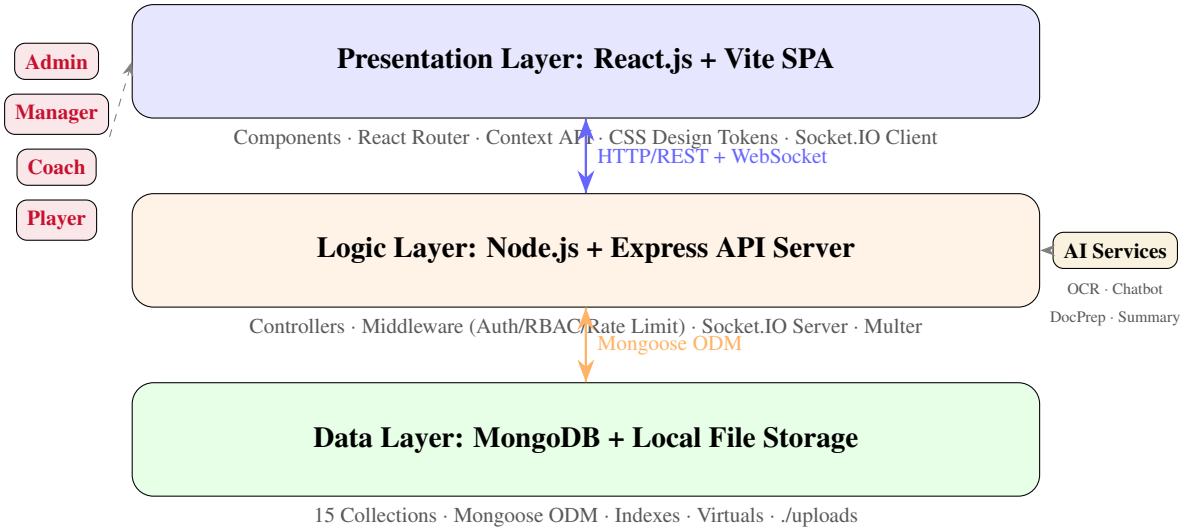

Figure 3.1: Three-Layer System Architecture

Key advantages of the proposed system include: a single source of truth for all club data, enterprise-grade security, AI-powered automation, real-time communication via WebSocket, a complete immutable audit trail, scalable MERN architecture, and a premium cinematic UI that promotes adoption.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The MERN stack is one of the most widely adopted and well-documented technology stacks. All libraries (Mongoose, Socket.IO, JWT, bcrypt, Multer, Helmet) are mature and production-proven. The technologies are fully capable of delivering real-time communication, AI capabilities, and comprehensive CRUD operations. **Verdict: Technically feasible.**

3.3.2 Economic Feasibility

The entire stack is open-source. MongoDB offers a free tier for development. Node.js, React, and all npm packages carry permissive licenses. Hosting is achievable on free-tier platforms. The long-term savings in operational efficiency outweigh the one-time development cost. **Verdict: Economically feasible.**

3.3.3 Operational Feasibility

The role-based interface ensures each user sees only relevant features, reducing cognitive load. The cinematic UI makes the system engaging compared to spreadsheet alternatives. Login is simple email/password authentication. Training time is minimal due to the intuitive panel design. **Verdict: Operationally feasible.**

CHAPTER 4

SYSTEM DESIGN

4.1 Data Flow Diagrams

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. It illustrates how data is processed in terms of inputs and outputs across the four user roles. Table 4.1 defines the standard DFD symbols used.

Table 4.1: DFD Symbol Definitions

Symbol	Name	Represents
Rectangle	External Entity	An outside actor (Admin, Manager, Coach, Player)
Circle / Oval	Process	A function performed by the system
Parallel Lines	Data Store	A repository (MongoDB collection)
Arrow	Data Flow	Direction and type of data movement

4.1.1 Level 0 — Context Diagram

Figure 4.1 shows the system as a single process with all four external entities.

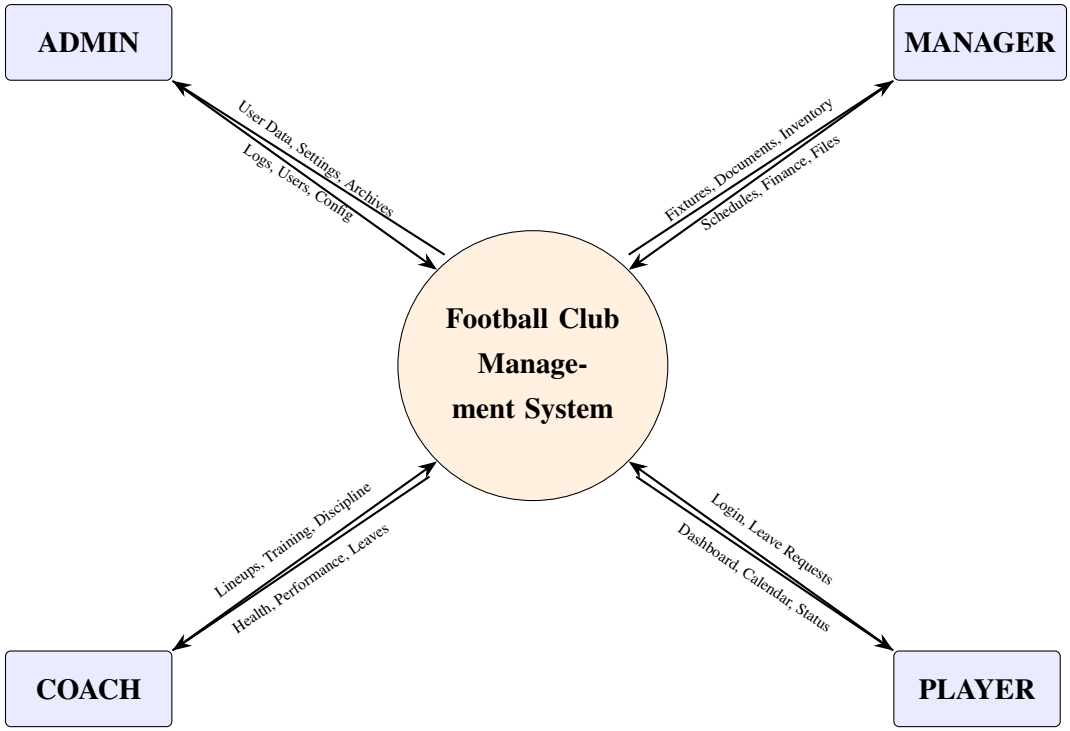

Figure 4.1: Level 0 — Context Diagram

4.1.2 Level 1 — Admin Module DFD

Figure 4.2 decomposes the Admin's interactions into sub-processes.

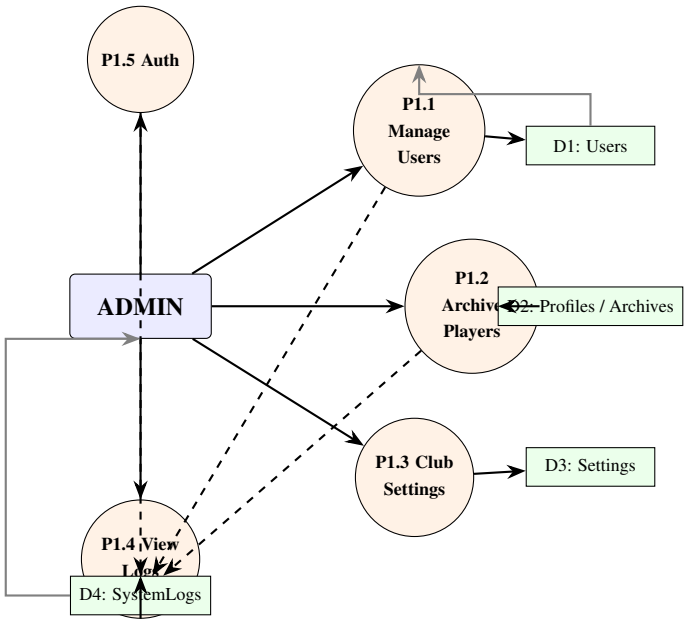

Figure 4.2: Level 1 — Admin Module DFD

4.1.3 Level 1 — Manager Module DFD

Figure 4.3 shows the Manager's sub-processes for scheduling and resource management.

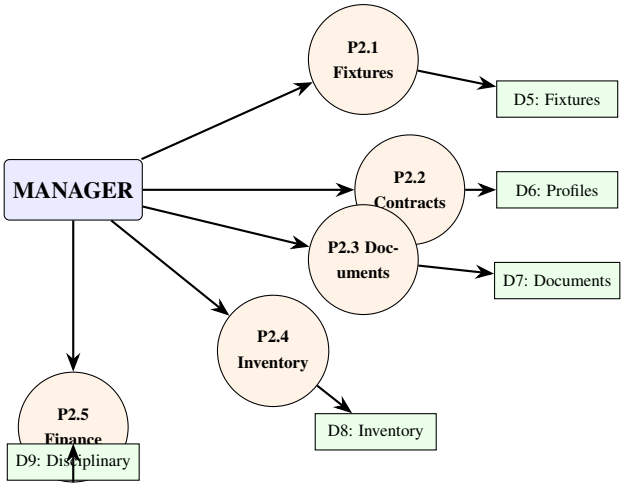

Figure 4.3: Level 1 — Manager Module DFD

4.1.4 Level 1 — Coach Module DFD

Figure 4.4 details the Coach's tactical and squad management processes.

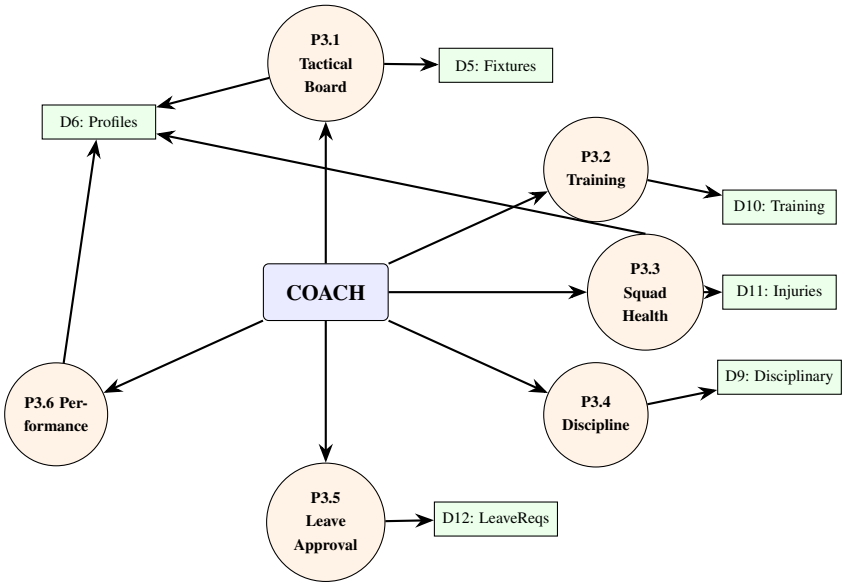

Figure 4.4: Level 1 — Coach Module DFD

4.1.5 Level 1 — Player Module DFD

Figure 4.5 shows the Player's view-and-submit processes.

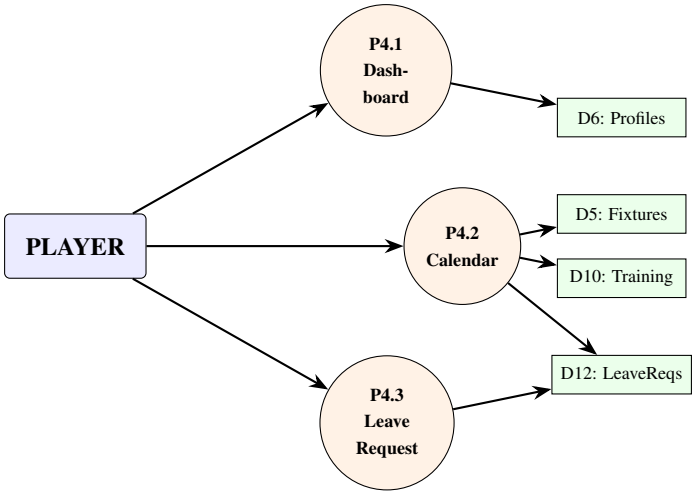

Figure 4.5: Level 1 — Player Module DFD

4.2 Entity Relationship Diagram

Figure 4.6 illustrates the relationships among the 15 MongoDB collections.

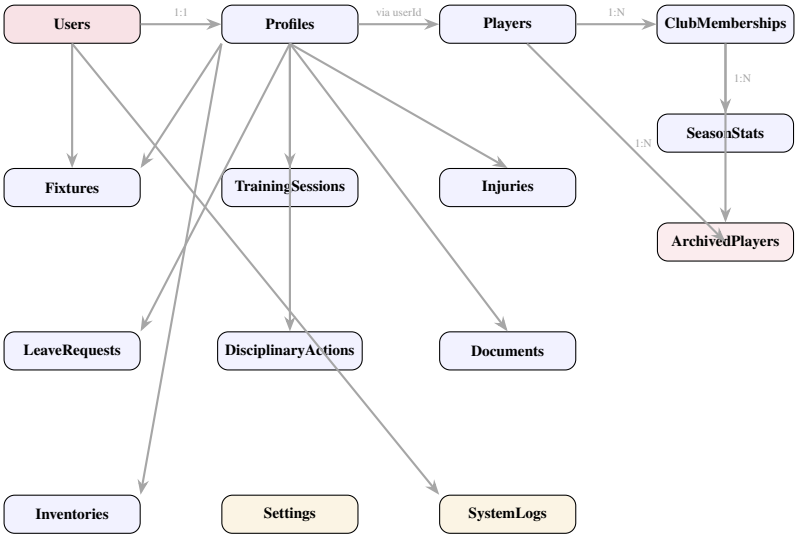

Figure 4.6: Entity Relationship Diagram — 15 Collections

CHAPTER 5

TABLE DESIGN

The system uses **15 MongoDB collections** with Mongoose ODM for schema enforcement, validation, virtuals, and pre/post hooks. MongoDB uses `_id` (ObjectId) as the auto-generated primary key for every document. The following sections detail each collection's structure.

5.1 Users Collection

Table 5.1: Users Collection Schema

#	Field	Type	Null	Constraints
1	_id	ObjectId	NO	Primary Key (auto)
2	email	String	NO	UNIQUE, lowercase, RFC 5322 regex
3	passwordHash	String	NO	bcrypt hashed; select:false
4	role	String (enum)	NO	IN: admin, manager, coach, player
5	createdAt	Date	NO	Immutable, default: Date.now

Indexes: `{role: 1}`

5.2 Profiles Collection

Table 5.2: Profiles Collection Schema (Key Fields)

#	Field	Type	Null	Constraints
1	userId	ObjectId (ref: User)	NO	UNIQUE, FK to Users
2	fullName	String	NO	2–100 chars
3	position	String (enum)	YES	GK, DEF, MID, FWD, Staff
4	fitnessStatus	String (enum)	YES	Green, Yellow, Red
5	contractType	String (enum)	YES	Full-Time, Part-Time, Loan, Trial
6	contractStart	Date	YES	—
7	contractEnd	Date	YES	Must be $\geq$ contractStart
8	stats.*	Number (sub-doc)	YES	goals, assists, appearances, rating, etc.
9	performanceNotes	Array of Sub-doc	YES	note, createdBy, createdAt

Indexes: `{fitnessStatus: 1}`, `{contractEnd: 1}`
Virtuals: `contractDaysRemaining`, `contractExpiringSoon` ($<$ 90 days)

5.3 Players Collection

Table 5.3: Players Collection Schema

#	Field	Type	Null	Constraints
1	fullName	String	NO	2–100 chars
2	dateOfBirth	Date	YES	—
3	nationality	String	YES	Max 60 chars
4	status	String (enum)	YES	active, archived, transferred, retired
5	currentUserId	ObjectId (ref)	YES	Sparse index

5.4 ClubMemberships Collection

Tracks a player's tenure, jersey number, position, contract, and availability. Key fields include `playerId` (FK to Players), `jerseyNumber` (1–99), `primaryPosition`, `contractType` (Owned/Loan/Academy/Trial/Staff), `squadRole` (starter/rotation/prospect/captain), and `availabilitySt` with detail sub-document.

Indexes: `{playerId:1, isActive:1}`, `{jerseyNumber:1, isActive:1}`, `{joinedAt:-1}`

5.5 SeasonStats Collection

Stores per-season statistics: goals, assists, yellowCards, redCards, minutesPlayed, appearances, offsides, and matchRatings (array of fixture-specific ratings with coach notes). Compound unique index on `{membershipId, season}`.

Virtuals: `averageRating` computed from matchRatings array.

5.6 ArchivedPlayers Collection

Preserves a frozen snapshot of a player upon archival. Contains `reason` (transferred/released/retired/etc.), `careerSummaryAtClub` (totalGoals, totalAssists, totalAppearances, seasonsPlayed, trophiesWon), and `snapshot` (fullName, photo, jerseyNumber, position at time of archival).

5.7 Fixtures Collection

Table 5.4: Fixtures Collection Schema

#	Field	Type	Null	Constraints
1	opponent	String	NO	2–100 chars
2	date	Date	NO	Cannot be in the past
3	location	String	NO	—
4	matchType	String (enum)	YES	League, Cup, Friendly, Tournament
5	lineup	Array of ObjectId	YES	Max 18 (11 starters + 7 subs)
6	lineupHistory	Array of Sub-doc	YES	version, savedAt, savedBy, formation

5.8 TrainingSessions Collection

Fields: `date` (future only), `drillDescription` (10–500 chars), `duration` (30–300 min), `attendees` (array of sub-docs with playerId and status: Present/Absent/Excused), `createdBy`.

5.9 Injuries Collection

Fields: `playerId` (FK), `injuryType`, `severity` (Minor/Moderate/Severe), `expectedRecovery`, `resolved` (boolean), `loggedBy`.

5.10 LeaveRequests Collection

Fields: `playerId`, `startDate`, `endDate` ($\geq$ startDate), `reason` (10–500 chars), `status` (Pending/Approved/Denied), `reviewedBy`, `reviewedAt`.

5.11 DisciplinaryActions Collection

Fields: `playerId`, `offense` (5–200 chars), `fineAmount` (0–100,000), `isPaid`, `paymentDate`, `issuedBy`.

5.12 Documents Collection

Fields: `playerId`, `fileName`, `originalName`, `filePath`, `fileType` (PDF/JPEG/PNG), `fileSize` (max 10 MB), `uploadedBy`.

5.13 Inventories Collection

Fields: `itemName`, `itemType` (Jersey/Boots/Training Equipment/Medical/Other), `assignedTo` (ref), `assignedAt`, `returnedAt`. Virtual: `isAssigned`.

5.14 Settings Collection (Singleton)

A single document storing: `clubName`, `logoUrl`, `homepageHeadline`, `clubDescription`, `founded`, `ground`, `league`, `contactEmail`, `socialHandle`, `trophies[]` (title+year). Uses `getSingleton()` static method.

5.15 SystemLogs Collection (Immutable)

Fields: `action` (CREATE/UPDATE/DELETE/LOGIN), `performedBy` (FK), `targetCollection`, `targetId`, `changes` (Mixed), `timestamp` (immutable). Pre-hooks prevent all update and delete operations—logs are write-once, read-many.

CHAPTER 6

MODULE DESIGN AND IMPLEMENTATION

The system consists of four role-based modules, a shared login page, and four AI-powered features. Each module is implemented as a React page component rendered within the `PanelShell` layout—a scrollable, single-page workspace with fixed side navigation, scroll-spy highlighting, and smooth section navigation.

6.1 Login Page

The Login Page is the gateway to the system. It features a full-screen, dark-themed layout with a centered glassmorphic card containing:

- Club crest badge with pulsing glow animation
- Email and password input fields with focus-state red border transitions
- "SIGN IN" button with hover lift effect and loading state
- Error alert banner for failed authentication

Authentication Flow: User submits credentials → AuthContext sends POST to `/api/auth/login` → Backend validates via bcrypt, generates JWT → SystemLog records LOGIN event → Frontend stores token, redirects to role-specific panel. Token verification runs on every page mount via `GET /api/auth/verify`.

6.2 Admin Module

Route: `/admin` Allowed Role: admin only

6.2.1 User Management

Full CRUD capabilities for all user accounts (admin, manager, coach, player) displayed in a table/grid format. Features inline creation form with email validation, role selector, and bcrypt-hashed password storage. Every mutation is logged to SystemLogs.

6.2.2 Player Archive

Manages the complete player lifecycle. Administrators can archive players with reason selection (transferred/released/retired/etc.) and notes, view archived players with frozen career snapshots (totalGoals, totalAssists, totalAppearances), and reinstate archived players.

6.2.3 Club Settings

Manages club identity through a singleton Settings document: club name, logo, headline, description, founded year, stadium, league, contact email, social handle, and dynamic trophies list.

6.2.4 System Logs

Read-only immutable audit trail with filter bar (action type, collection, date range, performer), chronologically sorted log table with color-coded action badges, and live presence indicator via Socket.IO.

6.3 Manager Module

Route: `/manager` Allowed Role: manager only

6.3.1 Fixtures

Fixture scheduling with opponent, date (future-only validation), location, and match type. Cards display upcoming and past fixtures.

6.3.2 Contracts

Contract lifecycle tracking with overview grid showing contract type, start/end dates, days remaining, and red-highlighted expiry alerts for contracts expiring within 90 days.

6.3.3 Document Vault

Secure file management with drag-and-drop upload (PDF/JPEG/PNG, max 10 MB), document grid with metadata, download links, and OCR scanner integration for text extraction.

6.3.4 Inventory

Equipment tracking with item creation, player assignment/return workflows, and status badges (Assigned/Available). Virtual `isAssigned` computed from assignment state.

6.3.5 Finance Dashboard

Disciplinary fine tracking with summary cards (total/paid/unpaid fines, payment rate), detailed fine table, and mark-as-paid functionality.

6.4 Coach Module

Route: `/coach` Allowed Role: coach only

6.4.1 Tactical Board

The largest and most complex component. Features fixture selector, formation dropdown (4-3-3, 4-4-2, 3-5-2, etc.), CSS-drawn pitch visualization with drag-and-drop player tokens (11 starters + 7 subs), player pool with position and fitness filters, lineup validation (max 18), and version history.

6.4.2 Training Schedule

Session creation with date, drill description (10–500 chars), duration (30–300 min), and per-player attendance marking (Present/Absent/Excused) with bulk capabilities.

6.4.3 Squad Health

Color-coded fitness overview (Green/Yellow/Red), injury logging with severity levels, injury history with resolution tracking, and manual availability override.

6.4.4 Discipline

Disciplinary action management with fine issuance (offense + amount), payment status tracking, and payment toggle.

6.4.5 Leave Approval

Pending request cards with Approve/Deny buttons and request history with status badges. Approval triggers real-time Socket.IO notification to the player.

6.4.6 Performance Tracking

Stat editor for goals, assists, appearances, minutes, cards, and rating (0–10). Performance notes timeline with coach attribution and timestamps.

6.5 Player Module

Route: `/player` Allowed Role: player only

6.5.1 Personalized Dashboard

Landing page with profile header, statistics grid (goals, assists, appearances, rating), contract information with expiry countdown, recent performance notes, and EA FC-style animated player card.

6.5.2 Calendar

Unified monthly calendar combining fixtures, training sessions, and approved leave dates with color-coded legend (Matches=red, Training=blue, Leave=green).

6.5.3 Leave Requests

Submit form with date pickers and reason textarea (10–500 chars). Request history shows status badges and review details.

6.6 AI-Powered Features

6.6.1 OCR Document Scanner

Extracts text from uploaded documents (PDF/image) for searching and indexing. Multi-page PDF support.

6.6.2 AI Document Preparation

Auto-generates standardized club documents from templates pre-filled with player data from multiple collections.

6.6.3 AI Chatbot

Natural language query interface accessible from any panel. Maps questions to database queries and returns human-readable responses.

6.6.4 Text Summarization

Condenses long document text into concise bullet-point summaries highlighting key facts, dates, and numbers.

CHAPTER 7

SECURITY MEASURES

The system implements a multi-layered security architecture. Table 7.1 summarizes all 17 security features.

Table 7.1: Security Features Summary

#	Feature	Technology	Description
1	JWT Authentication	jsonwebtoken v9.0.2	Bearer token in every request; contains {id, role} payload
2	Password Hashing	bcrypt v5.1.1	Salt rounds=10; select:false excludes hash from queries
3	RBAC	roleGuard middleware	requireRole() restricts routes to specific roles; returns 403
4	Frontend Protection	ProtectedRoute.jsx	Redirects unauthorized users to login
5	Security Headers	Helmet v7.1.0	CSP, X-Frame-Options, HSTS, X-Content-Type-Options
6	CORS Policy	cors v2.8.5	Restricted to specific client URL; credentials enabled
7	Rate Limiting	express-rate-limit	API: 100/15min; Auth: 5/15min; Uploads: 10/15min
8	Input Sanitization	sanitize-html + validator	XSS prevention; RFC 5322 email validation
9	Schema Validation	Mongoose validators	Type, min/max, enum, custom validators; 400 on failure
10	Immutable Audit Logs	SystemLog pre-hooks	Prevents UPDATE/DELETE; write-once, read-many
11	Password Exclusion	select:false + transform	Hash never in query results or serialized output
12	Compression	compression v1.7.4	Gzip-compressed API responses
13	Env Variable Security	dotenv v16.3.1	Secrets in git-ignored .env files

#	Feature	Technology	Description
14	File Upload Validation	Multer config	PDF/JPEG/PNG only; max 10 MB
15	Global Error Handler	errorHandler.js	Type-specific handling; stack traces only in dev
16	Socket.IO Auth	JWT on connection	WebSocket connections require valid tokens
17	404 Handler	notFoundHandler	Structured error; prevents info leakage

CHAPTER 8

SYSTEM TESTING

8.1 Unit Testing

Unit testing was performed using **Jest v29.7.0** with **fast-check v3.15.0** for property-based testing. Tests verify individual components in isolation:

- **Model Tests:** Valid document creation, validation failures, virtual computation, immutability enforcement, and pre-hook behavior for each Mongoose model.
- **Controller Tests:** Correct HTTP status codes, response body structure, database interaction, and edge-case error handling.
- **Middleware Tests:** Token validation, role-based access, rate limiting, and error formatting.
- **Frontend Tests:** Component rendering, state management, form validation, and animation states.

8.2 Integration Testing

Integration testing used **Jest + Supertest v6.3.3** to verify end-to-end flows:

- **API Channel:** Client sends request with JWT → Express processes → Mongoose queries MongoDB → JSON response returned. Verified CRUD operations generate corresponding SystemLog entries.
- **Real-Time Channel:** Player submits leave request → API saves to database → Socket.IO pushes notification to Coach → Coach approves → Player receives instant status update—all without page refresh.

8.3 Test Cases

Table 8.1 documents the 25 formal test cases.

Table 8.1: System Test Cases

ID	Module	Test Case	Expected Result	Status
TC-001	Auth	Valid login	200, JWT + role	Pass
TC-002	Auth	Invalid password	401, Invalid credentials	Pass
TC-003	Auth	Non-existent email	401, Auth failed	Pass
TC-004	Auth	Empty fields	400, Required fields	Pass
TC-005	Auth	No token	401, No auth header	Pass
TC-006	Auth	Expired token	401, Token expired	Pass
TC-007	Auth	Wrong role access	403, Access denied	Pass
TC-008	Admin	Create valid user	201, user object	Pass
TC-009	Admin	Duplicate email	400, Duplicate email	Pass
TC-010	Admin	Invalid role	400, Invalid role	Pass
TC-011	Manager	Past-date fixture	400, Cannot be past	Pass
TC-012	Manager	Valid fixture	201, fixture object	Pass
TC-013	Manager	Oversize upload	400, Size exceeds max	Pass
TC-014	Manager	Invalid file type	400, Invalid type	Pass
TC-015	Coach	Lineup > 18	400, Max 18 exceeded	Pass
TC-016	Coach	Training < 30 min	400, Min 30 minutes	Pass
TC-017	Coach	Invalid severity	400, Not valid severity	Pass
TC-018	Coach	Approve leave	200, Updated request	Pass
TC-019	Player	End before start	400, End $\geq$ start	Pass
TC-020	Player	Short reason	400, Min 10 chars	Pass
TC-021	Security	Modify SystemLog	500, Cannot modify	Pass
TC-022	Security	Delete SystemLog	500, Cannot delete	Pass
TC-023	Security	Rate limit	429, Too many attempts	Pass
TC-024	Settings	Short club name	400, Min 3 chars	Pass
TC-025	Profile	Invalid contract	400, End after start	Pass

8.4 User Acceptance Testing

UAT was conducted with representative users from each role. Table 8.2 summarizes the results.

Table 8.2: User Acceptance Testing Results

ID	Role	Scenario	Acceptance Criteria	Result
UAT-01	Admin	Create player account	User in list with correct role	Accepted

ID	Role	Scenario	Acceptance Criteria	Result
UAT-02	Admin	Archive retired player	Career summary preserved	Accepted
UAT-03	Admin	Update club identity	Homepage reflects changes	Accepted
UAT-04	Admin	Review audit logs	Login events with timestamps	Accepted
UAT-05	Manager	Schedule league match	Fixture appears for all roles	Accepted
UAT-06	Manager	Upload contract PDF	Document in vault with metadata	Accepted
UAT-07	Manager	Assign jersey	Inventory shows assignment	Accepted
UAT-08	Manager	View financial summary	Totals match fine records	Accepted
UAT-09	Coach	Build 4-3-3 lineup	11 starters, 7 subs on pitch	Accepted
UAT-10	Coach	Training + attendance	Percentages correct	Accepted
UAT-11	Coach	Log knee injury	Fitness status changes	Accepted
UAT-12	Coach	Approve leave request	Player sees Approved status	Accepted
UAT-13	Player	View dashboard	All stats and info correct	Accepted
UAT-14	Player	Submit leave request	Appears in coach's queue	Accepted
UAT-15	Player	View calendar	Events on correct dates	Accepted

CHAPTER 9

SCOPE FOR FUTURE ENHANCEMENT

The modular MERN-stack architecture with Socket.IO real-time communication is inherently scalable. The following high-value enhancements have been identified:

1. **Mobile Application** — A React Native app mirroring the web platform for on-the-go access.
2. **Advanced AI Analytics** — Machine learning for player performance prediction, injury risk assessment, and optimal lineup recommendation.
3. **Video Analysis Integration** — AI-powered match highlight extraction, heatmap generation, and movement tracking.
4. **Multi-Club / Multi-Season Support** — Managing multiple clubs with season-based data partitioning.
5. **Financial Module Expansion** — Salary management, transfer fees, sponsorship revenue, and full accounting.
6. **Biometric Integration** — Wearable device data (GPS, heart rate) feeding into the health module.
7. **Parent/Guardian Portal** — Limited-access view for youth academy player guardians.
8. **Multi-Language Support (i18n)** — Internationalization for diverse club demographics.
9. **Progressive Web App (PWA)** — Offline caching, push notifications, home screen installation.
10. **Third-Party API Integrations** — Automatic fixture imports, live scores, and league standings.

CHAPTER 10

CONCLUSION

The Football Club Management System was undertaken to solve the deep-seated inefficiencies inherent in traditional, spreadsheet-and-paper-based football club management. The project successfully delivered a centralized, web-based digital ecosystem comprising four interconnected modules—Admin Panel, Manager Panel, Coach Panel, and Player Panel—each providing role-specific tools built on the robust MERN stack.

The system's design achieves both functional completeness and technical excellence. A secure Node.js/Express API handles all business logic with JWT authentication and bcrypt password hashing. Socket.IO provides real-time WebSocket communication for instant notifications, online presence tracking, and live data synchronization. MongoDB with Mongoose ODM serves as the single source of truth across 15 structured collections.

Key achievements include the visual **Tactical Board** with drag-and-drop lineup building and version history; the **Squad Health Monitor** with color-coded fitness indicators; the EA FC-style **Animated Player Card** providing a cinematic digital identity; and the **Immutable Audit Trail** ensuring complete accountability.

The four AI-powered features—OCR scanning, chatbot, document preparation, and summarization—elevate the system beyond standard CRUD operations into a decision-support platform. Enterprise-grade security with 17 implemented measures protects all sensitive club data.

The system was validated through unit, integration, and user acceptance testing, with all 25 formal test cases passing successfully. The modular architecture provides a solid foundation for the future enhancements detailed in Chapter 9. In conclusion, the Football Club Management System is a comprehensive and successful project that effectively digitizes core club operations, enhances communication, ensures accountability, and delivers a premium user experience for all stakeholders.

BIBLIOGRAPHY

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2020.

[2] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2016.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson Education, 2016.

[4] J. W. Satzinger, R. B. Jackson, and S. D. Burd, *Systems Analysis and Design in a Changing World*, 7th ed. Cengage Learning, 2016.

[5] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[6] L. Richardson and S. Ruby, *RESTful Web Services*. O'Reilly Media, 2007.

[7] OpenJS Foundation, "Node.js Documentation." [Online]. Available: https://nodejs.org/docs/latest/api/. [Accessed: Apr. 2026].

[8] MongoDB, Inc., "MongoDB Manual." [Online]. Available: https://www.mongodb.com/docs/manual/. [Accessed: Apr. 2026].

[9] Express.js, "Express 4.x API Reference." [Online]. Available: https://expressjs.com/en/4x/api.html. [Accessed: Apr. 2026].

[10] Meta Platforms, Inc., "React Documentation." [Online]. Available: https://react.dev/. [Accessed: Apr. 2026].

[11] Automattic, Inc., "Mongoose ODM Documentation." [Online]. Available: https://mongoosejs.com/docs/. [Accessed: Apr. 2026].

[12] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015. [Online]. Available: https://tools.ietf.org/html/rfc7519.

[13] Socket.IO, "Socket.IO Documentation (v4)." [Online]. Available: https://socket.io/docs/v4/. [Accessed: Apr. 2026].

[14] OWASP Foundation, "OWASP Top Ten Web Application Security Risks." [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 2026].

[15] S. Van Kerckhoven, *Professional Football Club Management: Leadership for Commercial Success*. Routledge, 2020.